Field Hours

CSCE 350 Computer Architecture and Design

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

The performance equation

Time is instruction count times cycles per instruction times the clock period. Speed up a program by cutting one of those, and say which one. A compiler that removes instructions cuts the first. A pipeline that finishes more instructions per cycle cuts the second. A faster clock cuts the third, until the logic no longer fits in the period.

A benchmark that only reports time, without saying which factor moved, is not an architecture result. This equation is the one CSCE 614 will take apart.

Time, as a product.

Instructions -> Cycles per instruction -> Seconds per cycle

Two machines, one program

1. Machine A runs 1 billion instructions at 1 cycle each and a 1-nanosecond clock. Time is 1 second.

2. Machine B runs the same source as 1.2 billion instructions, at 0.8 cycles each, same clock. Time is 0.96 seconds.

3. B did more instructions and still finished sooner, because cycles per instruction fell enough. Count all three factors.

Problem 1. A new compiler cuts the instruction count by half and doubles the cycles per instruction. What happens to time, if the clock is unchanged?

Answer. It stays the same. Half the instructions times twice the cycles is the same product. Measure before you call the compiler a win.

Problem 2. The clock period drops from 1 ns to 0.5 ns, but a stall adds a cycle to every instruction that used to take one. What is the new time relative to the old, per instruction?

Answer. Each instruction now takes 2 cycles times 0.5 ns, which is still 1 ns. The faster clock was spent on the stall. Net time per instruction did not change.

Lecture 2 · 50 minutes

A pipeline

A pipeline splits the single-cycle path into stages and starts the next instruction before the previous one finishes. With five stages, five instructions can be in flight. The ideal throughput is one instruction per clock, after the pipe fills. The latency of one instruction is still five clocks.

Throughput and latency are different. A pipeline improves throughput. It does not make one instruction's result

appear in the first clock.

Five stages, ideal.

Fetch

Decode

Execute

Memory

Write back

Three independent adds

1. Clock 1: add A is fetched.

2. Clock 2: add A decodes, add B is fetched.

3. Clock 5: add A writes back. Adds B and C are still in the pipe. By clock 7 all three have written back. A single-cycle machine with a clock five times longer would still be on its first or second instruction, depending on how you count the long period. The comparison only holds if you state both clocks.

Problem 1. Ideal pipeline, five stages, no stalls. How many clocks to retire 100 instructions, counting the fill?

Answer. 104. Four clocks to fill the pipe, then one retirement per clock for 100 instructions. The fill is 4, not 5, because the first instruction retires on clock 5, which is the first of the 100.

Problem 2. Why can the ideal rate fail on the very next instruction after a load?

Answer. The next instruction may need the loaded value before the memory stage has produced it. That dependence is a stall, or a forward from a later stage. Independence was the assumption.

Lecture 3 · 50 minutes

Hazards

A data hazard is an instruction that needs a result that is not written yet. Forwarding passes the value from the stage that computed it, when that stage already has it. A load produces the value in the memory stage, which is often one clock too late for the next instruction. That leftover stall is a load-use hazard.

A control hazard is a branch. The fetch stage does not know the next address until the branch executes. The machine guesses, or it stalls. A wrong guess throws away the instructions fetched after the branch.

A load-use stall.

1. Load starts

2. The next instruction needs the value

3. The value is not ready

4. One stall, then forward

Load, then add

1. Load R1 from memory. The next instruction adds R1 and R2.

2. When the add is in decode, the load has not finished memory. Forwarding from the ALU does not help, because the value is not in the ALU.

3. Insert one bubble. On the next clock the load's memory result can forward. The add continues.

Problem 1. Add R1, R2 writes R1, and the next instruction reads R1, and there is no memory. Can forwarding remove the stall?

Answer. Often yes. The sum exists at the end of execute, and the next instruction needs it at the start of its execute. A forward path supplies it. A design without that path stalls instead. Look at the pipeline diagram before you claim either.

Problem 2. A branch is taken 10 percent of the time and you always predict not taken. When do you pay?

Answer. On the taken branches, 10 percent, you discard the wrongly fetched instructions and pay the refill. The 90 percent not-taken branches pay nothing extra under that predictor. A loop that is taken every time is the opposite case, and this predictor is then wrong almost always.

Lecture 4 · 50 minutes

A first cache

A cache line holds several consecutive words. The index picks the set. The tag says which memory line is currently there. A hit compares the tag. A miss fetches the line and may evict another. Direct-mapped means one line per set, so two hot addresses that share an index evict each other forever.

This is the undergraduate picture. CSCE 614 will add associativity and the three miss kinds. Here, compute one index and say whether the next address hits.

An address, split.

Tag

Index, picks the set

Offset, picks the byte in the line

Line of 16 bytes, 4 sets

1. Address 20. Offset is 20 mod 16 = 4. The line number is floor(20 / 16) = 1. Index is 1 mod 4 = 1.

2. Address 84. Line number floor(84 / 16) = 5. Index 5 mod 4 = 1. Same set as address 20.

3. In a direct-mapped cache those two lines cannot both stay. Touching them in turn misses every time after the first fill.

Problem 1. You double the line size and a sequential scan gets faster, but a program that touches one word per struct gets slower. Why both?

Answer. The scan uses the extra words the line brought in. The struct walk brings in words it never reads and evicts a line it did need. Line size is a bet on spatial locality.

Problem 2. Two addresses hit the same index and their tags differ. Is that a hit?

Answer. No. The index found the set. The tag mismatch means the set holds a different line. Direct-mapped, that is a miss. A second way in the set could still hold the tag you want. This course's cache has one way.

Lecture 5 · 50 minutes

Pages

A virtual address is what the program uses. A page table maps the page number to a physical frame. The page offset does not go through the table. A translation lookaside buffer caches recent translations. A miss in that buffer walks the table. A miss in the table, because the page is not resident, is a fault.

You need this picture before CSCE 611 talks about processes, and before CSCE 614 talks about the walker. Compute one page number by hand.

From a virtual address to a frame.

Split off the offset -> Look up the page -> Attach the frame -> The offset is unchanged

Page size 4096, address 8192

1. 8192 / 4096 = 2 exactly. Page number 2, offset 0.

2. If page 2 maps to frame 5, the physical address is 5 times 4096, plus 0.

3. Address 8193 is the same page, offset 1. The table is not consulted for the offset.

Problem 1. Why can two processes use address 8192 and not share a physical frame?

Answer. Each process has its own page table. The same virtual page number can point at different frames. Sharing is a choice, when both tables name the same frame.

Problem 2. The TLB holds page 2 and you switch processes without flushing it. What can go wrong?

Answer. The old translation is used for the new process. Address 8192 reaches the previous process's frame. A process switch updates or flushes the translations. Forgetting that is a protection bug.

Lecture 6 · 50 minutes

More than one core

Two cores with private caches can hold two copies of one address. A write on one core does not change the other copy unless a protocol says so. Coherence is that protocol. You do not implement MESI here. You stop using a shared counter from two cores and expecting both to see every increment without a lock or an atomic instruction.

CSCE 614's coherence lecture is the mechanism. This lecture is the reason it exists.

One address, two caches.

Core 0

Holds x = 1

Writes x = 2

Core 1

Still holds x = 1 until told

A later read can be stale

Two increments

1. x starts at 0 in memory. Each core loads 0 into its cache and adds 1.

2. Each stores 1. The memory ends at 1, not 2, if the second store overwrites the first.

3. An atomic increment, or a lock around the load and store, makes the second core wait until it can add to 1 and store 2.

Problem 1. Both cores only read x. Does coherence have to move the line?

Answer. No. Many readers of one line are the point of a shared state. A write is what forces the other copies to update or drop.

Problem 2. You added a lock and the program is correct but slower with eight cores than with one. What might still be wrong?

Answer. The lock serializes the increments, so the eight cores take turns. Correctness is not speedup. If the increments were independent counters that happened to share a cache line, split them. That is false sharing, which the graduate course measures.